

Lab3

Design and Simulation of Basic Logic Gates Using VHDL

Objectives

To design, implement, and simulate basic logic gates (AND, OR, NOT, NAND, NOR, XOR) using VHDL and verify their functionality through simulation.

Software and Hardware Requirements

Software

- ModelSim (Intel/Quarta or ModelSim PE/SE)

Hardware

- Standard PC/Laptop

Theory

Logic gates are the fundamental building blocks of digital systems. They perform basic logical functions on binary inputs to produce a binary output.

VHDL (VHSIC Hardware Description Language) is used to describe the behavior and structure of digital circuits.

AND Gate

The AND gate performs logical multiplication. The output is HIGH (1) only when **all inputs are HIGH**. It is commonly used in control systems where multiple conditions must be satisfied simultaneously.



Figure 1: Logic Symbol of AND Gate

OR Gate

The OR gate performs logical addition. The output is HIGH (1) when **any one or more inputs are HIGH**. It is widely used in alarm systems and decision-making circuits.

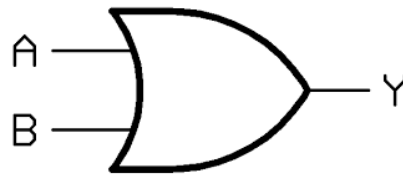


Figure 2: Logic Symbol of OR Gate

NOT Gate

The NOT gate is also called an inverter. It produces the **complement of the input**, converting logic 1 to 0 and logic 0 to 1. It is used for signal inversion and logic negation.

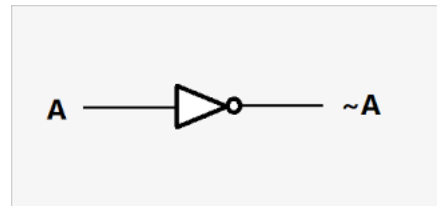


Figure 3: Logic Symbol of NOT Gate

NAND Gate

The NAND gate is the inverse of the AND gate. The output is LOW (0) only when **all inputs are HIGH**. NAND gates are called *universal gates* because any logic circuit can be implemented using only NAND gates.

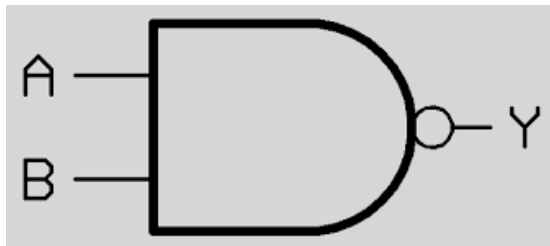


Figure 4: Logic Symbol of NAND Gate

NOR Gate

The NOR gate is the inverse of the OR gate. The output is HIGH (1) only when **all inputs are LOW**. Like NAND, NOR is also a *universal gate*.

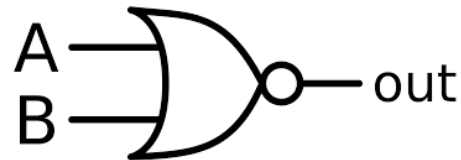


Figure 5: Logic Symbol of NOR Gate

XOR Gate

The XOR (Exclusive-OR) gate produces HIGH output when the **inputs are different**. It is commonly used in adders, parity generators, and error detection circuits.

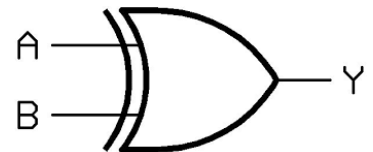


Figure 6: Logic Symbol of XOR Gate

Truth Tables

AND Gate

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

A	$Y = \bar{A}$
0	1
1	0

NAND Gate

A	B	$Y = (A \cdot B) \bar{}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

A	B	$Y = (A + B)̄$
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

VHDL Program Code

AND Gate

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity and_gate is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end and_gate;

architecture Behavioral of and_gate is
begin
```

```
    Y <= A and B;  
end Behavioral;
```

OR Gate

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity or_gate is  
    Port (  
        A : in STD_LOGIC;  
        B : in STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end or_gate;  
  
architecture Behavioral of or_gate is  
begin  
    Y <= A or B;  
end Behavioral;
```

NOT Gate

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity not_gate is  
    Port (  
        A : in STD_LOGIC;  
        Y : out STD_LOGIC  
    );  
end not_gate;  
  
architecture Behavioral of not_gate is  
begin  
    Y <= not A;  
end Behavioral;
```

NAND Gate

```
Y <= not (A and B);
```

NOR Gate

```
Y <= not (A or B);
```

XOR Gate

```
Y <= A xor B;
```

Observation

- AND Gate output is HIGH only when both inputs are HIGH.
- Similar verification is done for all logic gates.

Result

All basic logic gates were successfully designed using VHDL and their operations were verified through simulation. The observed waveforms matched the respective truth tables.

Discussion & Conclusion

This experiment successfully demonstrated the design and simulation of basic logic gates using VHDL. The lab enhanced understanding of digital logic design and VHDL-based modeling.